Goal of the Exercise

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

- The service will be a simple web server that returns a static page.
- You can use **apache2** or **nginx** to serve a simple HTML file.
- The service should be **containerized** and stored in a **Docker registry**.
- The service should be deployed on a **single VM instance** as a docker container.
- The image should be pulled from the registry specified earlier.
- The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

You are to provide the following as part of your solution:

1. **Dockerfile** to build the image.
2. **Terraform codebase** to deploy the following:
    - Docker registry
    - VPC and all required networking resources
    - EC2 instance
    - Load Balancer

Notes

- Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.
- To push the image to the Docker registry, **build the image locally** before running the Terraform code.
- Once the image registry is created, use Terraform to **push the local image** to the registry.
- Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.

- For SSH access:
    - Use a locally created key.
    - Add it to the cloud provider key storage.
    - Attach it to the instance.
- Use **Terraform best practices** for file structure and code organization.
    - Use variables, locals, modules, etc. when appropriate.
- Include all tfvars, scripts, and other helper files used to deploy the application.
- You may use any cloud provider of your choosing, although **AWS is preferred**.
- Use a **free tier account** or a **trial account** to deploy the resources.

Terraform Cloud Compute – Implementation Steps

1. Project Overview

The objective of this challenge was to deploy a static web application on AWS using Terraform, Docker, and AWS services.

The application was containerized using Docker, stored in Amazon ECR, and deployed on an Amazon EC2 instance. An Application Load Balancer (ALB) was configured to make the application accessible through the internet.

Technologies Used

- Terraform – Infrastructure as Code
- Docker – Containerization
- AWS ECR – Docker image repository
- AWS VPC – Network configuration
- AWS EC2 – Compute service
- AWS IAM – Access management
- AWS Application Load Balancer – Traffic distribution
- Amazon Linux 2023 – Operating system
- Nginx – Web server

2. Project Directory Structure

Created the following project directory structure in VS Code:

```
terraform-cloud-compute/
│
├── app/
│   ├── Dockerfile
│   └── index.html
│
└── terraform/
    ├── versions.tf
    ├── provider.tf
    ├── variables.tf
    ├── terraform.tfvars
    ├── locals.tf
    ├── ecr.tf
    ├── vpc.tf
    ├── security-groups.tf
    ├── keypair.tf
    ├── iam.tf
    ├── ec2.tf
    ├── user_data.sh
    ├── alb.tf
    └── outputs.tf
```

3. Created the Web Application

Created an HTML file named index.html inside the app directory.

The file contains the application welcome message.

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Terraform Cloud Compute</title>
</head>
<body>
  <h1>Welcome to Terraform Cloud Compute!</h1>
  <p>This application is running inside a Docker container on AWS EC2.</p>
  <p>Deployment: Terraform + Docker + AWS ECR + ALB</p>
</body>
</html>
```

4. Created the Dockerfile

Created a file named Dockerfile inside the app directory.

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Explanation:

- Used the lightweight Nginx Alpine image as the base image.
- Copied the HTML application into the Nginx web directory.
- Exposed port 80.
- Started Nginx in the foreground.

5. Built and Tested the Docker Image Locally

Opened PowerShell in the project root directory and executed:

```
docker build -t terraform-webapp:v1 ./app
```

Started a container for local testing:

docker run -d --name terraform-web-test -p 8085:80 terraform-webapp:v1

Tested the application using:

Invoke-WebRequest http://localhost:8085

The application returned HTTP 200 successfully.

After testing, removed the temporary container:

docker rm -f terraform-web-test

6. Configured Terraform Providers and Versions

Created versions.tf to specify the required Terraform version and providers.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.100"
    }

    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}
```

Created provider.tf to configure AWS and Docker providers.

```
provider "aws" {
  region = var.aws_region
```

```
}

provider "docker" {
  registry_auth {
    address  = split("/", aws_ecr_repository.app.repository_url)[0]
    username = data.aws_ecr_authorization_token.token.user_name
    password = data.aws_ecr_authorization_token.token.password
  }
}
```

Configured AWS in the us-east-1 region.

7. Created Terraform Variables

Created variables.tf to define reusable input variables.

```
variable "aws_region" {
  description = "AWS region for deployment"
  type        = string
  default     = "us-east-1"
}

variable "project_name" {
  description = "Project name"
  type        = string
  default     = "terraform-webapp"
}

variable "my_ip" {
  description = "Your public IP address in CIDR notation"
  type        = string
```

```
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "key_name" {
  description = "Name of the EC2 SSH key pair"
  type        = string
  default     = "terraform-web-key"
}
```

Created terraform.tfvars to provide the variable values.

```
aws_region    = "us-east-1"
project_name  = "terraform-webapp"
my_ip         = "<YOUR_PUBLIC_IP>/32"
instance_type = "t3.micro"
key_name      = "terraform-web-key"
```

The EC2 instance type was changed to t3.micro after AWS rejected the initial t2.micro launch as not eligible for the account's Free Tier.

8. Created Terraform Locals and Common Tags

Created locals.tf to define reusable local values and common resource tags.

```
locals {
  name = var.project_name

  common_tags = {
    Project     = local.name
```

```
  Environment = "dev"
  ManagedBy   = "Terraform"
 }
}
```

These tags were used across AWS resources to identify the project and environment.

9. Created the AWS ECR Repository

Created ecr.tf to create an Amazon Elastic Container Registry repository and push the Docker image.

```
resource "aws_ecr_repository" "app" {
 name                 = "${local.name}-repo"
 image_tag_mutability = "MUTABLE"

 image_scanning_configuration {
  scan_on_push = true
 }

 tags = local.common_tags
}

data "aws_ecr_authorization_token" "token" {}

resource "docker_image" "app" {
 name = "${aws_ecr_repository.app.repository_url}:v1"

 build {
  context    = "../app"
  dockerfile = "Dockerfile"
 }
```

```
}


resource "docker_registry_image" "app" {
  name = docker_image.app.name


  keep_remotely = true
}
```

Terraform performed the following operations:

- Created the ECR repository.
- Built the Docker image using the application directory.
- Authenticated with the ECR registry.
- Pushed the Docker image to ECR.

The image was successfully pushed with the tag v1.

10. Created the VPC and Networking Resources

Created vpc.tf to configure the AWS network.

VPC

Created a VPC with CIDR block 10.0.0.0/16.

Public Subnets

Created two public subnets in different Availability Zones.

- Public Subnet 1: 10.0.1.0/24
- Public Subnet 2: 10.0.2.0/24

Enabled automatic public IP assignment.

Internet Gateway

Created and attached an Internet Gateway to the VPC.

Route Table

Created a public route table with a default route to the Internet Gateway.

Route Table Associations

Associated both public subnets with the public route table.

The networking configuration was:

```
resource "aws_vpc" "main" {
  cidr_block         = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = merge(local.common_tags, {
    Name = "${local.name}-vpc"
  })
}

data "aws_availability_zones" "available" {
  state = "available"
}

resource "aws_subnet" "public_1" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true

  tags = merge(local.common_tags, {
    Name = "${local.name}-public-subnet-1"
  })
}
```

```
resource "aws_subnet" "public_2" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.2.0/24"
  availability_zone       = data.aws_availability_zones.available.names[1]
  map_public_ip_on_launch = true

  tags = merge(local.common_tags, {
    Name = "${local.name}-public-subnet-2"
  })
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = merge(local.common_tags, {
    Name = "${local.name}-igw"
  })
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = merge(local.common_tags, {
```

```
  Name = "${local.name}-public-rt"
 })
}

resource "aws_route_table_association" "public_1" {
 subnet_id     = aws_subnet.public_1.id
 route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_2" {
 subnet_id     = aws_subnet.public_2.id
 route_table_id = aws_route_table.public.id
}
```

11. Created Security Groups

Created security-groups.tf with two security groups.

ALB Security Group

Allowed inbound HTTP traffic from the internet on port 80.

EC2 Security Group

Configured the following inbound rules:

- SSH – Port 22, only from my public IP.
- HTTP – Port 80, only from the ALB security group.

Allowed outbound traffic for the EC2 instance and ALB.

```
resource "aws_security_group" "alb_sg" {
 name       = "${local.name}-alb-sg"
 description = "Allow HTTP traffic to ALB"
 vpc_id     = aws_vpc.main.id
```

```
  ingress {
    description = "Allow HTTP from internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = local.common_tags
}

resource "aws_security_group" "ec2_sg" {
  name        = "${local.name}-ec2-sg"
  description = "Allow SSH from my IP and HTTP from ALB only"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "SSH from my public IP"
    from_port   = 22
    to_port     = 22
```

```
  protocol    = "tcp"
  cidr_blocks = [var.my_ip]
 }

 ingress {
  description     = "HTTP from ALB only"
  from_port       = 80
  to_port         = 80
  protocol        = "tcp"
  security_groups = [aws_security_group.alb_sg.id]
 }

 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }

 tags = local.common_tags
}
```

This configuration ensures that users access the application through the ALB instead of directly through the EC2 public IP.

12. Created an SSH Key Pair

Generated an SSH key pair locally using PowerShell.

The private and public keys were stored in:

C:\Users\<USERNAME>\.ssh\

Created keypair.tf to upload the public key to AWS.

```
resource "aws_key_pair" "web" {
  key_name   = var.key_name
  public_key = file(pathexpand("~/.ssh/terraform-web-key.pub"))

  tags = local.common_tags
}
```

The private key was kept securely on the local machine and was not uploaded to AWS.

13. Created IAM Role and Instance Profile

Created iam.tf to allow the EC2 instance to access the ECR repository and pull the Docker image.

Created:

- IAM Role.
- Trust policy allowing EC2 to assume the role.
- AmazonEC2ContainerRegistryReadOnly policy attachment.
- IAM Instance Profile.

```
resource "aws_iam_role" "ec2_ecr_role" {
  name = "${local.name}-ec2-ecr-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
      Action = "sts:AssumeRole"
```

```
    }]
  })

  tags = local.common_tags
}

resource "aws_iam_role_policy_attachment" "ecr_readonly" {
  role      = aws_iam_role.ec2_ecr_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

resource "aws_iam_instance_profile" "web" {
  name = "${local.name}-instance-profile"
  role = aws_iam_role.ec2_ecr_role.name
}
```

14. Created EC2 User Data Script

Created user_data.sh to automate the application deployment when the EC2 instance starts.

The script pehrforms the following operations:

1. Updates the operating system.
2. Installs Docker and AWS CLI.
3. Starts anates wit Amazon ECR.
1. Pulls the Docker image.
2. Runs the appd enables Docker.
3. Authenticlication container on port 80.

```
#!/bin/bash

set -euxo pipefail


dnf update -y

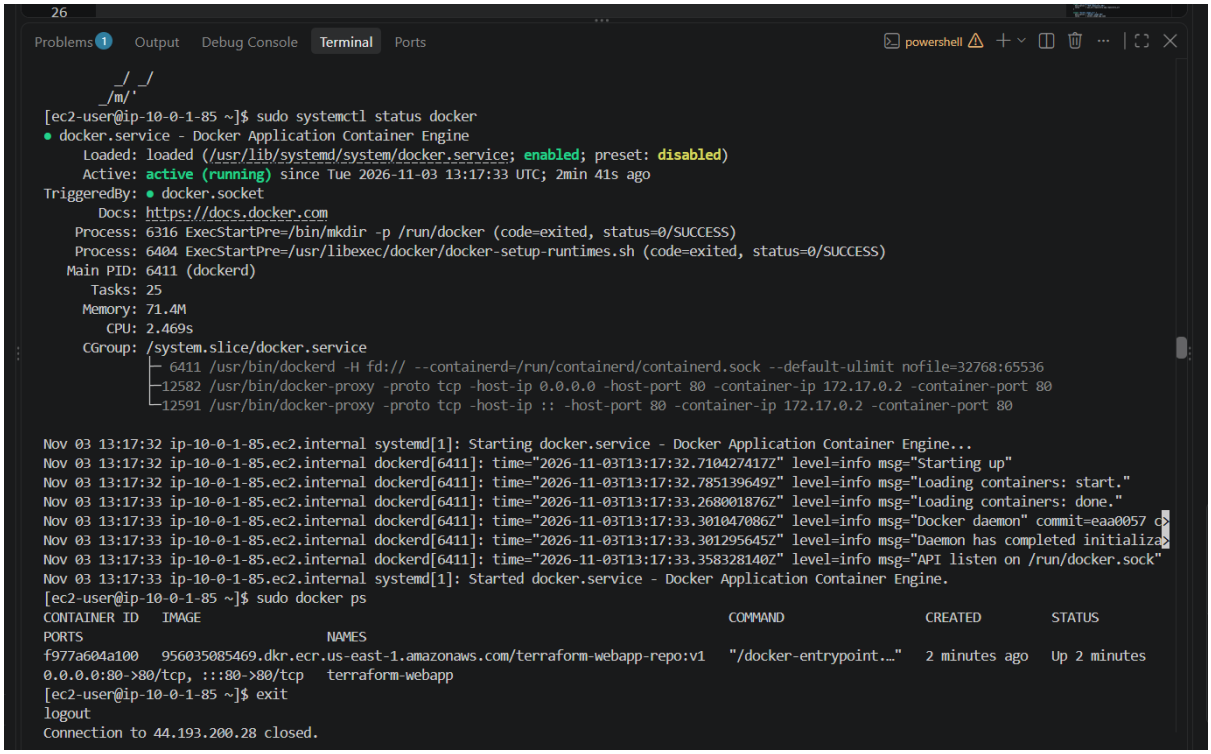
dnf install -y docker awscli


systemctl enable --now docker


aws ecr get-login-password --region ${aws_region} | docker login --username AWS --password-stdin ${registry}


docker pull ${image_uri}


docker run -d \
  --name terraform-webapp \
  --restart unless-stopped \
  -p 80:80 \
  ${image_uri}
```



15. Created the EC2 Instance

Created ec2.tf to launch an Amazon Linux 2023 EC2 instance.

Configured:

- Amazon Linux 2023 AMI.
- EC2 instance type using a Terraform variable.
- Public subnet.
- EC2 security group.
- SSH key pair.
- IAM instance profile.
- Public IP address.
- User data script.

```
data "aws_ami" "al2023" {
 most_recent = true
 owners      = ["amazon"]


 filter {
```

```
  name   = "name"
  values = ["al2023-ami-2023.*-x86_64"]
 }

 filter {
  name   = "architecture"
  values = ["x86_64"]
 }

 filter {
  name   = "virtualization-type"
  values = ["hvm"]
 }
}

resource "aws_instance" "web" {
 ami                    = data.aws_ami.al2023.id
 instance_type          = var.instance_type
 subnet_id              = aws_subnet.public_1.id
 vpc_security_group_ids = [aws_security_group.ec2_sg.id]
 key_name               = aws_key_pair.web.key_name
 iam_instance_profile   = aws_iam_instance_profile.web.name
 associate_public_ip_address = true

 user_data = templatefile("${path.module}/user_data.sh", {
  aws_region = var.aws_region
  registry   = split("/", aws_ecr_repository.app.repository_url)[0]
  image_uri  = docker_image.app.name
```

```
  })

  user_data_replace_on_change = true

  tags = merge(local.common_tags, {
    Name = "${local.name}-ec2"
  })
}
```

Initialized and validated the Terraform configuration.

Executed:

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

After reviewing the execution plan, approved the deployment by entering yes.

The EC2 instance was created successfully.

16. Verified EC2 Instance

Used AWS CLI to verify the instance details.

aws ec2 describe-instances --instance-ids <INSTANCE_ID> --region us-east-1 --query "Reservations[0].Instances[0].[InstanceId,State.Name,InstanceType,PublicIpAddress,PrivateIpAddress]" --output table

The EC2 instance was running successfully.

Connected to the instance using SSH:

ssh -i "$env:USERPROFILE\.ssh\terraform-web-key" ec2-user@<EC2_PUBLIC_IP>

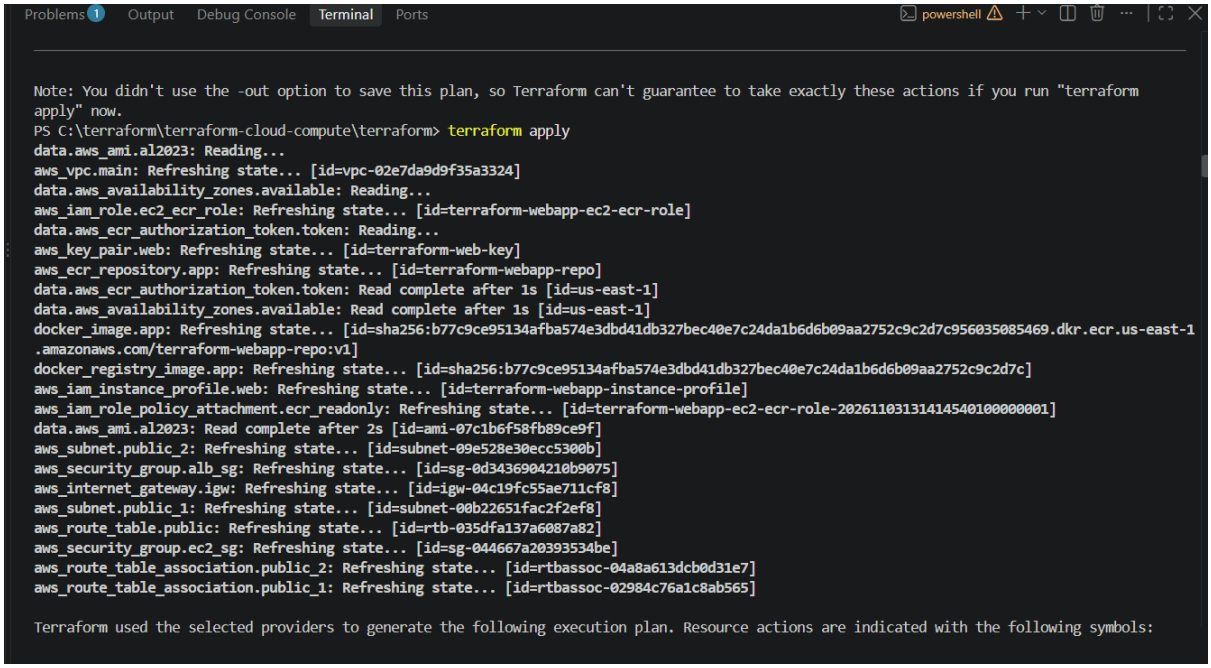
Checked Docker service:

sudo systemctl status docker

Checked the running container:

sudo docker ps

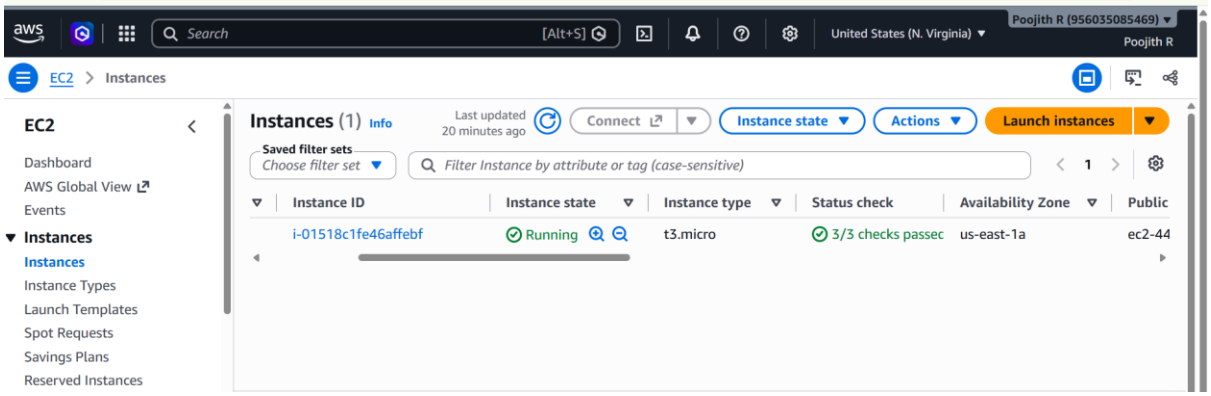
Verified that the terraform-webapp container was running and port 80 was mapped correctly.

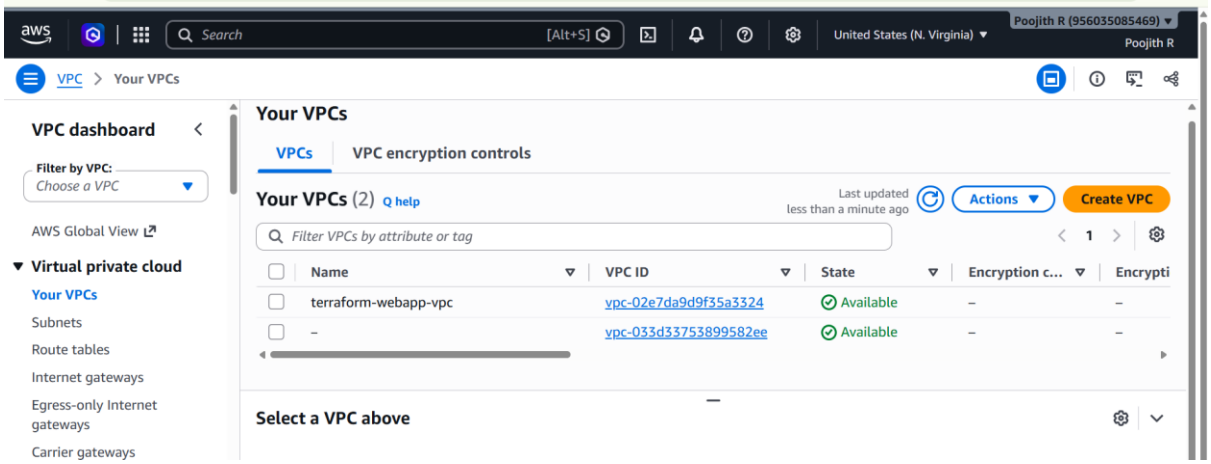


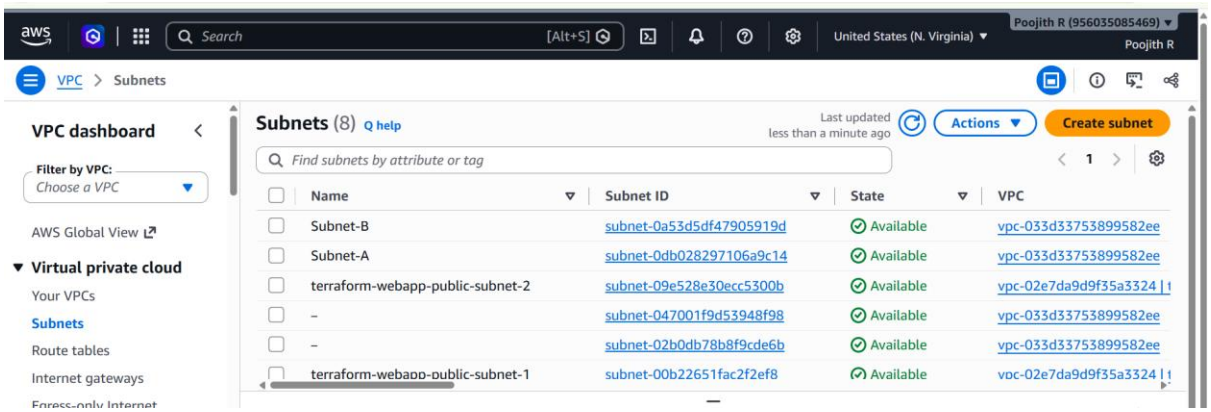
```
Problems 1  Output  Debug Console  Terminal  Ports                                   powershell

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform\terraform-cloud-compute\terraform> aws ec2 describe-instances --instance-ids i-01518c1fe46affebf --region us-east-1 --query "Reservations[0].Instances[0].[InstanceId,State.Name,InstanceType,PublicIpAddress,PrivateIpAddress]" --output table
------------------------
|   DescribeInstances  |
+----------------------+
|  i-01518c1fe46affebf |
|  running             |
|  t3.micro            |
|  44.193.200.28       |
|  10.0.1.85           |
+----------------------+

PS C:\terraform\terraform-cloud-compute\terraform> aws ec2 describe-instance-status --instance-ids i-01518c1fe46affebf --region us-east-1 --include-all-instances --output table
-------------------------------------------------------------------
|                    DescribeInstanceStatus                       |
+-----------------------------------------------------------------+
||                      InstanceStatuses                         ||
|+-----------------+---------------------+-----------------------+|
|| AvailabilityZone | AvailabilityZoneId  |      InstanceId       ||
|+-----------------+---------------------+-----------------------+|
||  us-east-1a      |  use1-az1           |  i-01518c1fe46affebf  ||
|+-----------------+---------------------+-----------------------+|
|||                     InstanceState                           |||
||+-------------------------+-------------------------------------+||
|||          Code           |                Name                 |||
||+-------------------------+-------------------------------------+||
|||  16                     |  running                            |||
||+-------------------------+-------------------------------------+||
```







17. Created the Application Load Balancer

Created alb.tf to configure the Application Load Balancer.

Created four resources:

1. Application Load Balancer.
2. Target Group.
3. Target Group Attachment.
4. HTTP Listener.

```
resource "aws_lb" "app" {
 name            = "${local.name}-alb"
 internal        = false
 load_balancer_type = "application"

 security_groups = [aws_security_group.alb_sg.id]

 subnets = [
  aws_subnet.public_1.id,
  aws_subnet.public_2.id
 ]

 tags = local.common_tags
}
```

```
resource "aws_lb_target_group" "app" {
  name     = "${local.name}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id

  health_check {
    path     = "/"
    protocol = "HTTP"
    matcher  = "200"
  }

  tags = local.common_tags
}

resource "aws_lb_target_group_attachment" "app" {
  target_group_arn = aws_lb_target_group.app.arn
  target_id        = aws_instance.web.id
  port             = 80
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
```

```
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

Executed:

terraform fmt

terraform validate

terraform plan

terraform apply

Terraform successfully created all four ALB resources.

18. Verified ALB and Target Group Health

Retrieved the ALB DNS name using AWS CLI.

aws elbv2 describe-load-balancers --names terraform-webapp-alb --region us-east-1 --query "LoadBalancers[0].[DNSName,State.Code]" --output table

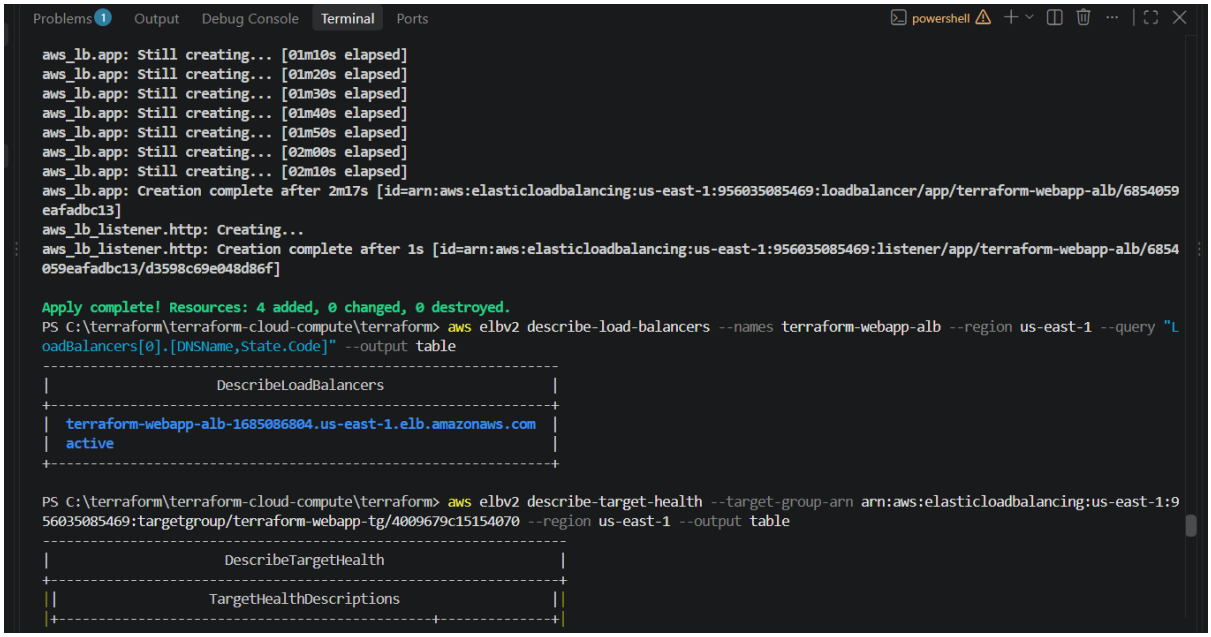
Verified the target group health:

aws elbv2 describe-target-health --target-group-arn <TARGET_GROUP_ARN> --region us-east-1 --output table

The target health was confirmed as:

healthy

The ALB was in the active state.



19. Tested the Web Application

Opened the ALB DNS URL in a web browser.

Application URL:

http://terraform-webapp-alb-1685086804.us-east-1.elb.amazonaws.com

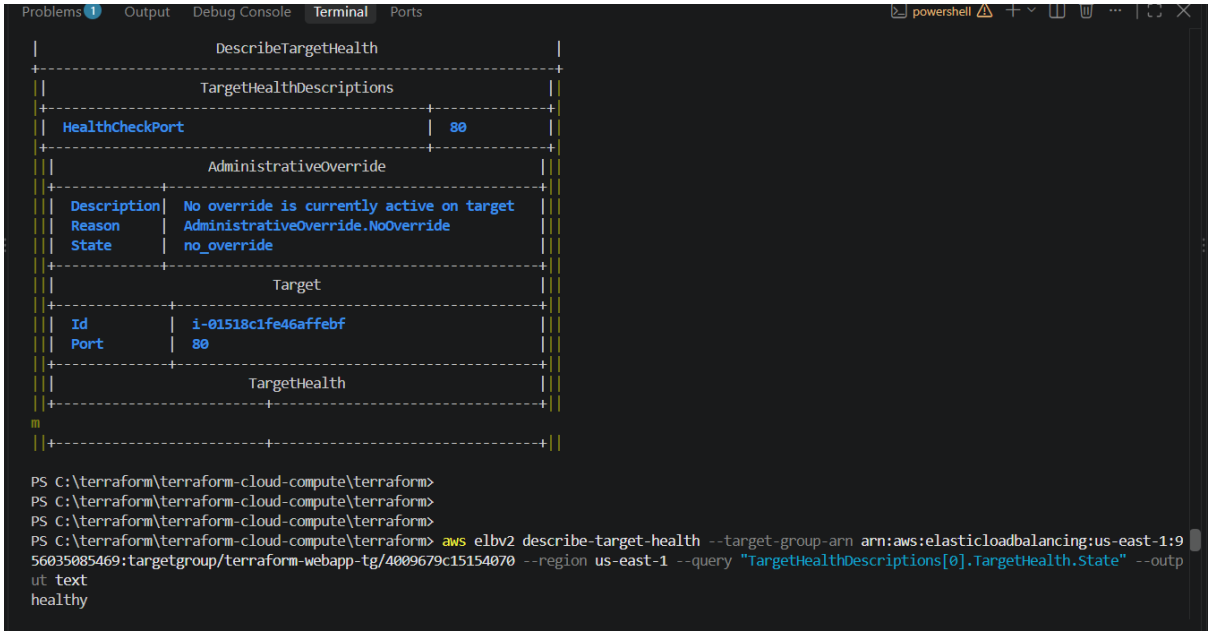
The following application content was displayed successfully:

Welcome to Terraform Cloud Compute!

This application is running inside a Docker container on AWS EC2.

Deployment: Terraform + Docker + AWS ECR + ALB

This confirmed that the application was successfully deployed and accessible through the Application Load Balancer.



20. Created Terraform Outputs

Created outputs.tf to display important deployment information.

```
output "alb_dns_name" {
 description = "Application Load Balancer DNS name"
 value       = aws_lb.app.dns_name
}


output "application_url" {
 description = "Web application URL"
 value       = "http://${aws_lb.app.dns_name}"
}


output "ec2_public_ip" {
 description = "EC2 public IP address for SSH access"
 value       = aws_instance.web.public_ip
}


output "ecr_repository_url" {
```

```
 description = "ECR repository URL"
 value       = aws_ecr_repository.app.repository_url
}

output "docker_image_uri" {
 description = "Docker image URI"
 value       = docker_image.app.name
}
```

Executed:

terraform fmt

terraform validate

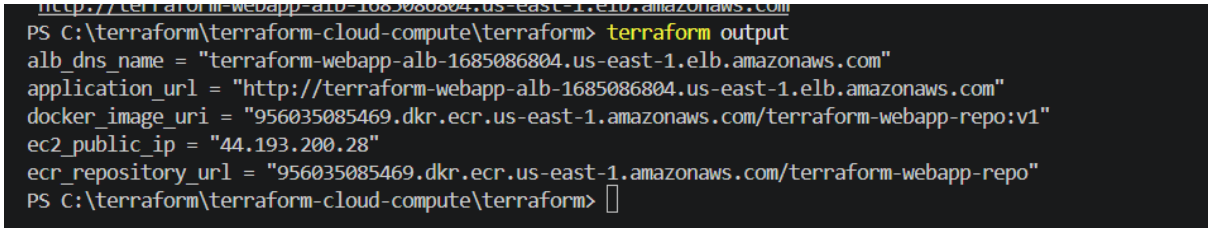
terraform plan

terraform apply

terraform output

Terraform saved and displayed the output values successfully.

No infrastructure changes were required during this output configuration.



21. Final Result

Successfully deployed the web application using Terraform, Docker and AWS.

The final architecture was:

Internet → Application Load Balancer → Target Group → EC2 Instance → Docker Container → Nginx Web Application.

The following components were successfully verified:

- Docker image built and pushed to ECR.
- VPC and networking resources created.

- Security groups configured.
- IAM role and instance profile created.
- EC2 instance launched successfully.
- Docker installed and running on EC2.
- Application container running successfully.
- Application Load Balancer created.
- Target group health confirmed as healthy.
- Website accessed successfully through ALB DNS.
- Terraform outputs generated successfully.

Final Status: Terraform Cloud Compute challenge successfully implemented and verified.

